Contents

Bachelor Thesis: Fine-tuned Frequent-Hitters in Drug Discovery

Github Link

Khaled Awadallah - K11945506

September 2025

Abstract

Drug discovery is an expensive process, often hampered by a lack of experimental data, which hinders the application of conventional machine learning techniques to this problem domain. Few-shot learning methods attempt to address the problem of data scarcity by building predictive models from few available data samples. While advanced meta-learning methods like Model-Agnostic Meta-Learning (MAML) are the current state-of-the-art in few-shot learning, their practical application can be computationally costly and complex. This thesis investigates a simpler approach, the "Fine-tuned Frequent Hitters" method, by training a neural network to predict the average activity of each molecule across different tasks and fine-tuning it on new tasks with only a few training data points. Using the MUV benchmark dataset, the proposed method achieved a mean ROC AUC score of 0.7136 ± 0.0290 and a mean DAUPRC score of 0.0747 ± 0.0313 on unseen test tasks. For comparison, MAML with 5 inner gradient steps achieved a mean ROC AUC of 0.6749 ± 0.0431 and a mean DAUPRC of 0.0717 ± 0.0347, while MAML with 10 inner gradient steps achieved a mean ROC AUC of 0.6832 ± 0.0450 and a mean DAUPRC of 0.0798 ± 0.0372. Fine-tuned Frequent Hitters outperforms both MAML variants on the ROC AUC metric, whereas MAML with ten gradient steps attains the highest DAUPRC. These results indicate that a simple fine-tuning strategy yields better overall performance on the ROC AUC metric, while deeper inner-loop adaptation in MAML can improve precision-recall behavior in highly imbalanced settings. The findings suggest that well-implemented fine-tuning approach provides a solid and simpler alternative to more complex meta-learning approaches such as the MAML algorithm.

1 Introduction

In the field of medicine, it is essential to continuously work to discover new drugs in order to combat diseases and enhance human health. However, drug discovery is an extremely complex and time-consuming process that involves

identifying molecules that can interact with specific molecular targets in the body to treat diseases [1]. On average, bringing a new drug to the market can take 10-15 years and cost billions of dollars, with a low rate of success at every stage. This makes it difficult for companies and researchers to sustain their efforts in any drug discovery project.

Recently, Machine Learning-based approaches have proven to be beneficial in shortening the time and cost of developing new drugs by analyzing vast datasets to predict potential drug candidates, and identify drug targets. Today, machine learning is used in all stages of drug development, especially in the initial stages of finding candidate molecules for further testing. However, machine learning methods require large amounts of training data, which can be expensive or even unavailable in many drug discovery projects. Therefore, it is still important to develop methods that can deal with the low-data problem.

Few-Shot Learning [2, 3, 4] is an area of machine learning that deals with the problem of few available data samples. The goal of few-shot learning is to build models that can generalize effectively from a very small number of training examples and achieve reasonable predictive performance. This approach can be promising in a field like drug discovery where known molecular activity is scarce.

One leading algorithm within Few-shot learning is Model-Agnostic Meta-Learning (MAML) which aims to train a model that can quickly adapt to a new task using only a few data samples and gradient updates. Meta-Learning methods like MAML, while powerful, introduce considerable complexity, such as requiring gradients of gradients, which can make them more difficult to train and tune. In my thesis, I explore a simpler and more straightforward alternative, which is a transfer learning approach based on training a Frequent Hitters model that learns the average activity of each molecule across different tasks and then fine-tuning it for specific tasks using a few data points as training examples.

The central research question of my thesis is therefore: Can a straightforward fine-tuned Frequent Hitters model compete with, or even outperform, the performance of a state-of-the-art meta-learning algorithm like MAML for few-shot learning in drug discovery ?

To answer this question, I implement and evaluate four different models on the challenging Maximum Unbiased Validation (MUV) benchmark dataset: a standard Random Forest baseline, a baseline Frequent Hitters model that neglects the support set, the state-of-the-art MAML algorithm, and the proposed Fine-tuned Frequent Hitters model. The performance is measured using the Area Under the ROC Curve (ROC AUC) and the Delta Area Under the Precision-Recall Curve (DAUPRC).

This thesis will detail the methodology of each approach, compare their performance, and discuss the results and implications of the findings.

2 Background and Related Work

2.1 Drug Discovery

The process of drug discovery involves the intricate task of identifying candidate molecules. This entails the screening of large libraries of molecules to identify those that interact with a specific biological target, a protein or an enzyme, in order to produce a desired therapeutic effect. As highlighted in the introduction, this is an extremely costly and time-consuming process that can take more than a decade and billions of dollars with an extremely high failure rate.

Machine Learning offers a powerful approach to accelerating the initial stages of this process by predicting the bioactivity of molecules. Commonly framed as a Quantitative Structure-Activity Relationship (QSAR) problem, this challenge aims at building a computational model to predict a molecule's biological activity based on its structure [5]. In a typical QSAR task, the model tries to predict whether or not a molecule is 'active' (likely to interact with the target) or 'inactive' (unlikely to interact with the target) for a particualar biological assay, or task.

2.2 Molecular Representation for Machine Learning

To apply machine learning to QSAR problems, a molecule's structural information must be translated into a numeric format that an algorithm can process. This is achieved by **molecular representations**, which are feature vectors that encode a molecule's structure and chemical properties [6]. The representations used in this thesis are:

- **Extended-Connectivity Fingerprints (ECFP)**: ECFPs are bit vectors that describe the local atomic environment within a molecule [7]. Each bit of the fingerprint corresponds to presence or absence of a specific small substructure, thus capturing the compound's detailed topology.
- **Molecular Descriptors**: Molecular descriptors are computed physicochemical properties of the molecule, such as molecular weight, polarity, or solubility [8]. They are a more general, property-based description of the compound.

By combining these two sets of features, the model is presented with a compact, fixed-length vector that captures both the structural details and the overall chemical nature of each molecule.

2.3 Few-Shot Learning in Drug Discovery

In standard supervised machine learning [9], a dataset $\mathcal{D} = \{(x_k, y_k)\}_{k=1}^{n}$ is usually split into $\mathcal{D}^{train} = \{(x_k, y_k)\}_{k=1}^{t}$ and $\mathcal{D}^{test} = \{(x_k, y_k)\}_{k=t+1}^{n}$, where n denotes the number of data samples and t denotes the number of training

samples. A model is trained on the training set $\mathcal{D}^{train}$ and evaluated by testing its predictions on the test set $\mathcal{D}^{test}$. This paradigm is highly effective when large training datasets are available, and when both the training and test data are drawn from the same distribution. However, in the context of drug discovery, data availability is often limited, and large-scale training sets may not exist.

Few-shot learning addresses this challenge by leveraging knowledge acquired from pre-training on different but related tasks and then adapting the model to a target task with only a small number of labeled samples [10]. Few-shot learning methods can broadly be categorized into three groups. **Data-augmentation-based approaches** expand the available dataset by artificially generating new, more diverse samples [11, 12]. **Embedding-based and nearest-neighbor approaches** focus on learning meaningful representations in an embedding space. Predictions for new samples can then be made by comparing these embeddings. For instance, Matching Networks [13] employ an attention mechanism over embeddings to guide predictions, while Prototypical Networks [14] compute a prototype representation for each class in the embedding space. **Optimization-based approaches** aim to learn a model initialization that can be adapted to new tasks by performing gradient descent steps. The proposed Fine-tuned Frequent Hitters method in this thesis belongs to the optimization-based approach, since it also performs task-specific fine-tuning through gradient descent updates on a small support set. This similarity in the adaptation mechanism provides the rationale for selecting MAML as a benchmark method for comparison, as both approaches leverage gradient-based fine-tuning to specialize a pre-trained model to new tasks.

In the context of drug discovery, the objective is to develop an effective predictive model from a limited set of molecules $X = \{x_1, ..., x_N\}$ paired with their corresponding activity measurements $y = \{y_1, ..., y_N\}$, where $y_n \in \{0, 1\}$ typically denotes inactive and active molecules, respectively. This dataset, denoted $Z = \{X, y\}$, is referred to as the support set.

A general optimization-based few-shot learning method can be seen as a model that uses the support set Z to update the parameters w to predict $\hat{y}$ for a query molecule m

$$\hat{y} = g_{a(w;Z)}(m) \tag{1}$$

where $a(.)$ represents an adaptation function, which is a gradient-based update in the context of this thesis.

2.4 Baseline Frequent Hitters

Baseline Frequent Hitters g^{FH} [15] is a naive baseline that uses the usual few-shot learning process, which involves evaluating a specific query molecule m, paired with its label y. However, the frequent hitters model g^{FH} completely

neglects the support set Z:

$$\hat{y} = g_w^{FH}(m) \tag{2}$$

Practically, the model tries to predict the average activity of a molecule across all trained tasks to minimize loss, since the model might predict both $y = 1$ and $y = 0$ during training for the same molecule m considering that the molecule can be active in one task and inactive in another task.

2.5 Fine-tuned Frequent Hitters

Fine-tuned Frequent Hitters is the main method proposed in this thesis. It is an extension of the baseline where a neural network is pre-trained to predict the average activity of a molecule in all trained tasks, generating a set of pre-trained weights w_{pre}, containing the generalized knowledge that the model has learned. Then, for each new target task, the model is fine-tuned by incorporating the support set Z. This adaptation updates the weights by performing gradient descent steps to minimize the loss on the task-specific data:

$$w \leftarrow w_{pre} - \alpha \nabla_{w_{pre}} \mathcal{L}_{z \in Z}(g(z)) \tag{3}$$

where $\mathcal{L}$ is a loss function that measures the error on g, and α is the learning rate.

2.6 Model-Agnostic Meta-Learning (MAML)

Model-Agnostic Meta-Learning (MAML) [16] is a state-of-the-art few-shot learning approach that has a fundamentally different objective from traditional machine learning. Instead of learning a model's parameters that directly solve one task, MAML learns an optimal parameter initialization θ which is highly sensitive and can be rapidly adapted to new, unseen tasks with very few data samples (*see Figure 1*).

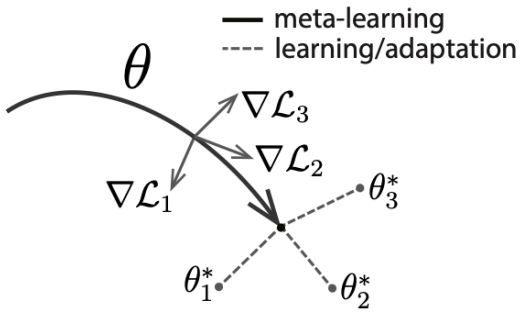

Figure 1: Diagram of model-agnostic meta-learning algorithm (MAML), which optimizes for a representation θ that can quickly adapt to new tasks.

This process is often called "learning to learn" and is achieved through a nested optimization approach (*see Algorithm 1*). During the meta-training phase, the algorithm samples a batch of tasks $\mathcal{T}$. For each individual task $\mathcal{T}_i$ in the batch, the algorithm performs an inner loop update by simulating a learning process on the small support set of the task. It calculates adapted parameters $\theta_i^{'}$ by taking one or few gradient descent steps with respect to the task's loss function $\mathcal{L}_{\mathcal{T}_i}$. For one gradient step, this update is defined as

$$\theta_i^{'} = \theta - \alpha \nabla_\theta \mathcal{L}_{\mathcal{T}_i}(f_\theta) \tag{4}$$

where f_θ is the model parameterized by θ and α is the inner-loop learning rate. Following this adaptation, the algorithm performs an outer loop update to improve the initial meta-parameters θ. The performance of the adapted model $f_{\theta_i^{'}}$ is evaluated on the task's query set. The loss from this evaluation is subsequently used to update the initial meta-parameters θ. This meta-update is provided by

$$\theta \leftarrow \theta - \beta \nabla_\theta \sum_{\mathcal{T}_i \sim p(\mathcal{T})} \mathcal{L}_{\mathcal{T}_i}(f_{\theta_i^{'}}) \tag{5}$$

where β is the meta-learning rate. The most important part of MAML is that the meta-optimization gradient ∇_θ is calculated by differentiating through the inner-loop adaptation step, a process often called a "gradient through a gradient". By repeating this process over many tasks, the parameters θ converge to an optimal initialization. In the final meta-testing phase, this learned initialization θ is adapted to a new test task with its corresponding support set, and the final adapted model is used for prediction.

Algorithm 1 Model-Agnostic Meta-Learning

Require: $p(\mathcal{T})$: distribution over tasks
Require: α, β: step size hyperparameters
1: randomly initialize θ
2: **while** not done **do**
3: Sample batch of tasks $\mathcal{T}_i \sim p(\mathcal{T})$
4: **for all** $\mathcal{T}_i$ **do**
5: Evaluate $\nabla_\theta \mathcal{L}_{\mathcal{T}_i}(f_\theta)$ with respect to K examples
6: Compute adapted parameters with gradient descent: $\theta_i' = \theta - \alpha \nabla_\theta \mathcal{L}_{\mathcal{T}_i}(f_\theta)$
7: **end for**
8: Update $\theta \leftarrow \theta - \beta \nabla_\theta \sum_{\mathcal{T}_i \sim p(\mathcal{T})} \mathcal{L}_{\mathcal{T}_i}(f_{\theta_i'})$
9: **end while**

3 Experiments

This section details the empirical framework designed to evaluate the central research question whether a straightforward fine-tuned model can compete with a state-of-the-art meta-learning algorithm in a few-shot drug discovery context. First, I describe the general experimental setup, including the data split, feature representation, and evaluation process. Subsequently, the conceptual basis and specific training and evaluation procedures for each of the four models compared in this study are presented.

3.1 Experimental Setup

To ensure a rigorous and unbiased comparison, a standardized experimental protocol was established for all models.

3.1.1 Dataset and Pre-processing

The dataset used in this thesis is the MUV dataset [17], which consists of 93,087 unique molecules and 17 different tasks [18]. One task (task 17) was excluded from this analysis because experiments indicated that its inclusion consistently degraded the performance of all models tested. The remaining 16 tasks were then split into three distinct sets: 10 tasks for training, 3 tasks for validation (used for hyperparameter optimization), and 3 tasks for testing. This separation guarantees that the final model performance is evaluated on tasks that were entirely unseen during both training and hyperparameter tuning. During preprocessing, missing labels (NaN entries) in the label matrix are excluded and the label matrix is then restructured into a triplet format (mol_id, target_id, label), where each triplet corresponds to one molecule-target pair. This representation facilitates efficient retrieval of pre-computed molecular features from mol_id during data loading. The resulting label distributions are highly imbalanced across all splits with a positive rate of approximately 0.2% in each split (see *table 1*), highlighting the strong class imbalance of the benchmark and motivating the use of DAUPRC in addition to ROC AUC as evaluation metrics.

Split	**Inactives**	**Actives**	**Total**	**% actives**
Training (10 tasks)	146,713	289	147,002	0.20%
Validation (3 tasks)	43,914	88	44,002	0.20%
Test (3 tasks)	44,048	88	44,136	0.20%

Table 1: Label distribution across training, validation, and test splits of the MUV dataset.

Note that although the dataset comprises 93,087 unique molecules, each molecule can be associated with multiple tasks. As a result, after restructuring into molecule–task–label triplets, the training, validation, and test splits contain more samples than unique molecules in the dataset.

3.1.2 Molecular Representation

Each of the 93,087 unique molecules in the dataset is converted into a fixed-size feature vector of 2,248 dimensions. This vector is a concatenation of 2,048 Extended Connectivity Fingerprints (ECFP) and 200 pre-selected molecular descriptors, capturing both structural and physicochemical properties. All features are standardized by subtracting the mean and scaling to unit variance.

3.1.3 Few-Shot Task Formulation

For the validation and test phases, each task is structured as a distinct few-shot learning problem. For a given task, the available molecule-activity pairs are divided into (a) a **support set**, created by randomly sampling 5 active (positive) and 5 inactive (negative) molecules, and (b) a **query set**, comprising all remaining molecules from that task. The small, 10-sample support set is used for task-specific adaptation, and the query set is used exclusively for evaluating the model's predictive performance on unseen data points within that task.

3.1.4 Evaluation

All experiments are carried out over 10 independent runs using different random seeds to ensure the robustness of the results. The performance of each model is assessed using two primary metrics: the Area Under the Receiver Operating Characteristic Curve (ROC AUC) and the Delta Area Under the Precision-Recall Curve (DAUPRC).

- **ROC AUC**: The ROC AUC metric [19] measures the area under the Receiver Operating Characteristic (ROC) curve, which plots the true positive rate (sensitivity) against the false positive rate (specificity) at various classification thresholds. This score provides an aggregate measure of performance across all possible threshold values. It ranges from 0 to 1 with 0.5 reflecting random guessing. The ROC AUC score is a robust metric that reflects the model's ability to discriminate between positive and negative classes, where a higher ROC AUC score indicates better performance in differentiating between the two classes.

- **DAUPRC**: The DAUPRC metric [20] is derived from the Area Under the Precision-Recall Curve (AUPRC), which plots *precision* (ratio of true positives to all positive predicictions) against *recall* (ratio of true positives to all actual positives) [21]. DAUPRC adjusts the AUPRC by subtracting the baseline performance of a random classifier, i.e., $\text{DAUPRC} = \text{AUPRC} - \text{AUPRC}_{\text{baseline}}$, thereby quantifying the improvement over random guessing. This adjustment makes the metric particularly informative in imbalanced classification settings, where a high baseline precision may occur by chance.

3.2 Methods

Four distinct models were implemented and evaluated, representing a spectrum from standard machine learning baselines to few-shot learning methods.

Random Forest (Baseline): Random Forest is a common machine learning algorithm that combines the outputs of multiple decision trees to reach a single prediction [22]. This approach is particularly effective in handling overfitting and provides strong performance across variety of tasks. This process directly measures the performance of a standard machine learning algorithm in a low-data environment, providing a crucial reference point for the few-shot learning methods. For each of the three test tasks, a new independent Random Forest classifier is trained from scratch, and for a given test task, the model is trained exclusively on the task's 10-sample support set. The trained classifier is then immediately evaluated on the corresponding query set. This process directly measures the performance of a standard machine learning algorithm in a low-data environment.

Baseline Frequent Hitters: The baseline frequent hitters model is a neural network designed to learn, a generalized, task-agnostic representation of molecular activity. The underlying hypothesis is that by training on a diverse set of tasks simultaneously, the model can learn fundamental structure-activity relationships that are broadly applicable. For training, a single feed-forward neural network is trained on an aggregated dataset composed of all available molecule-activity pairs from the 10 training tasks (147,002 total pairs). In this setup, each pair is treated as an independent training sample, and the model is not given explicit information about which task a sample belongs to. The training objective is to learn a general mapping from molecular features to a molecule's average activity across all tasks. Then, the single, trained model is evaluated directly on the query sets of the three test tasks. Crucially, this model performs its evaluation without any task-specific adaptation. The 10-sample support set for each test task is completely ignored during this phase, providing a performance baseline for a generalized model that does not leverage task-specific data at test time. Details on the method and hyperparameters selection are discussed in *Appendix A.1*

Fine-tuned Frequent Hitters: This model, the primary method proposed in this thesis, is a classic transfer learning approach. It extends the baseline frequent hitters model by incorporating a task-specific adaptation phase to specialize its generalized knowledge for new, unseen tasks. The first phase is pre-training , which is similar to the training of the baseline frequent hitters model, where a dedicated feed-forward neural network is pre-trained on the aggregated data from the 10 training tasks. This results in a pre-trained model, containing a generalized understanding of structure-activity relationships. The second phase is fine-tuning, where for each of the three test tasks, the model is fine-tuned exclusively on the task's 10-sample support set for 300 episodes. This adaptation

step allows the model to adjust its generalized weights to the specific patterns of the new task. The resulting specialized model is then evaluated on the task's query set. Details on the method and hyperparameters selection are discussed in *Appendix A.2*. Note that the fine-tuning process is done exclusively on the validation set and the best hyperparameters are chosen. The test set is only used for the final performance evaluation.

Model-Agnostic Meta-Learning (MAML): MAML is a state-of-the-art meta-learning algorithm designed not to learn to solve any single task, but rather to find an optimal parameter initialization that can be rapidly adapted to new tasks with very few examples. This process is often described as "learning to learn".

During the meta-training phase, a new MAML model is meta-trained using the 10 training tasks, where in each meta-training episode, a meta-batch of tasks is randomly sampled, and for each of these tasks, a 5-shot support set and a corresponding query set are created, by randomly sampling five positive and five negative samples to form the support set, and the rest of the samples form the query set. The parameters of the model are then updated via a nested optimization loop: (a) an **Inner Loop (Adaptation)**, where the current parameters of the model get temporarily updated by performing a few gradient descent steps on the task's support set, and an **Outer Loop (Meta-Update)**, where the performance of these temporary, adapted parameters are evaluated on the task's query set. The loss from this evaluation is then used to update the original parameter initialization, optimizing it to be a better starting point for future adaptations. During the testing phase, the final meta-learned parameter initialization is used as the starting point. For each of the three test tasks, this initial model is adapted by performing a few gradient descent steps on that specific task's 10-sample support set. The number of gradient steps is an important design choice, as the core premise of Model-Agnostic Meta-Learning (MAML) is to find an optimal initial set of parameters that can rapidly adapt to a new task with only one or a few gradient steps. Adhering to this fundamental assumption, I initially used five inner-loop gradient steps during the adaptation phase for each test task. This choice is consistent with the established practice and is within the typical range suggested by the authors of the original MAML paper. To thoroughly explore a potential improvement in performance through slightly deeper fine-tuning, I also conducted a separate experiment where the MAML model was adapted using ten gradient steps. This approach provides a comprehensive view of MAML's capabilities in both the standard few-shot adaptation setting (5 steps) and a slightly extended adaptation setting (10 steps). This is contrasted with the Fine-tuned Frequent Hitters model, which, lacking a meta-learned initialization, inherently requires more training steps to achieve competitive performance. Finally, the specialized model is evaluated on the corresponding test query set. Details on the method and hyperparameters selection are discussed in *Appendix A.3*

4 Results

This section presents the empirical findings of the comparative experiments on the unseen test tasks of the MUV dataset. The performance of the four models, namely Random Forest, Baseline Frequent Hitters, MAML, and the proposed Fine-tuned Frequent Hitters, is evaluated using the ROC AUC and DAUPRC metrics. Each experiment is carried out ten times independently with different random seeds. For each seed, the mean ROC AUC and DAUPRC scores are computed across all 3 test tasks, and finally the mean and standard deviation across all seeds are reported. The results are clearly presented in *Table 2* and illustrated in *Figure 2*.

	Random Forest		**Baseline Frequent Hitters**		**Fine-tuned Frequent Hitters**		**MAML (5 grad steps)**		**MAML (10 grad steps)**	
Seed	roc auc	dauprc	roc auc	dauprc	roc auc	dauprc	roc auc	dauprc	roc auc	dauprc
0	0.7430	0.0472	0.6508	0.0015	0.6892	0.0800	0.6782	0.0964	0.6662	0.1075
1	0.6724	0.0063	0.6202	0.0008	0.7492	0.0330	0.7176	0.0339	0.7226	0.0440
2	0.7611	0.0241	0.6650	0.0016	0.6742	0.0573	0.5750	0.0262	0.5942	0.0311
3	0.5241	0.0157	0.6202	0.0008	0.6831	0.0760	0.6656	0.0473	0.6682	0.0662
4	0.7215	0.0156	0.6749	0.0019	0.7117	0.0670	0.6758	0.0718	0.6725	0.0631
5	0.6376	0.0059	0.6216	0.0009	0.7470	0.0773	0.7202	0.0922	0.7353	0.1005
6	0.6726	0.0178	0.6369	0.0011	0.7027	0.0992	0.6669	0.0930	0.6533	0.1132
7	0.6282	0.0418	0.6966	0.0021	0.6870	0.0231	0.6309	0.0259	0.6513	0.0222
8	0.7401	0.0514	0.6572	0.0012	0.7421	0.0974	0.6979	0.0982	0.7217	0.1287
9	0.6476	0.0308	0.6479	0.0014	0.7501	0.1371	0.7204	0.1323	0.7468	0.1213
Mean	0.6748	0.0257	0.6491	0.0013	0.7136	0.0747	0.6749	0.0717	0.6832	0.0798
Std	0.0675	0.0156	0.0241	0.0004	0.0290	0.0313	0.0431	0.0347	0.0450	0.0372

Table 2: Performance comparison of all models with mean and standard deviation across 10 random seeds. Each reported score is the mean score across all 3 test tasks

Random Forest classifier, as a baseline machine learning algorithm, achieved a mean ROC AUC score of 0.6748 ± 0.0675 and a mean DAUPRC score of 0.0257 ± 0.0156. Its performance on the ROC AUC metric is respectable, but is modest on the DAUPRC metric. Furthermore, the high standard deviation, particularly for the ROC AUC metric, indicates considerable performance variation (see *Figure 2*).

The baseline frequency hitters model, which is a generalized neural network without task-specific tuning, achieved a mean ROC AUC score of 0.6491 ± 0.0241. However, its mean DAUPRC score of 0.0013 ± 0.0004 was extremely poor. This can be a significant finding: while the model has learned an overall representation that allows it to distinguish positive and negative classes better than random guessing (ROC AUC > 0.5), its performance on the DAUPRC metric is extremely poor, reflecting the metric's sensitivity to data imbalance. This result highlights the inadequacy of a generalized approach and the need

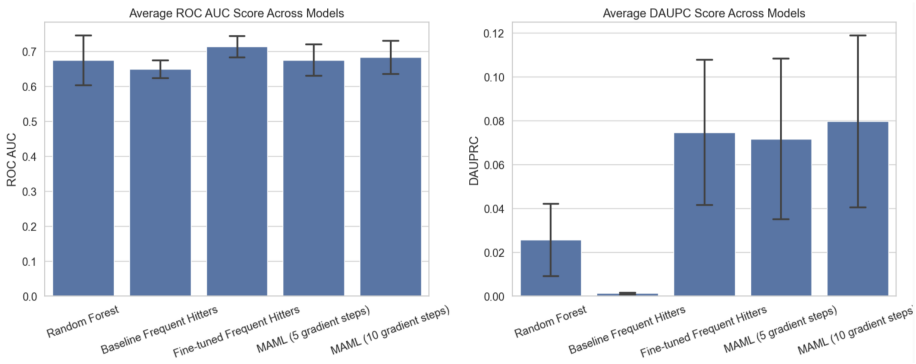

Figure 2: Average performance comparison of all models on the unseen test tasks. The barplots display the Mean ROC AUC (left) and Mean DAUPRC (right) scores. Each bar represents the mean performance across all seeds and test tasks, and the vertical error bars indicate the standard deviation calculated across the 10 independent random seeds.

for task-specific fine-tuning.

The MAML algorithm, being one of the state-of-the-art methods in meta-learning, was evaluated with two inner-loop configurations. When using 5 inner gradient steps, MAML achieved a mean ROC AUC of 0.6749 ± 0.0431 and a mean DAUPRC of 0.0717 ± 0.0347. Increasing the number of inner gradient steps to 10 produced a mean ROC AUC of 0.6832 ± 0.0450 and a mean DAUPRC of 0.0798 ± 0.0372. These results show that the additional inner-loop adaptation in MAML improves its DAUPRC and slightly improves ROC AUC compared to the 5-step configuration. Generally, the high DAUPRC score demonstrates that the MAML algorithm was able to find a better initialization of the parameters θ that can adapt to new tasks in a few gradient descent steps. However, MAML struggled on the ROC AUC metric, which despite the complexity of the MAML's algorithm, was not able to perform better than a simple baseline model such as Random Forest.

The proposed Fine-tuned Frequent Hitters model achieved a mean ROC AUC of 0.7136 ± 0.0290 and a mean DAUPRC of 0.0747 ± 0.0313. Comparing the methods, Fine-tuned Frequent Hitters yields the highest ROC AUC, outperforming both MAML variants on this metric. On the DAUPRC metric, however, MAML with 10 inner steps attains the best mean performance (0.0798), exceeding the Fine-tuned Frequent Hitters (0.0747). The standard deviations show that Fine-tuned Frequent Hitters is relatively more stable on ROC AUC (std = 0.0290) compared to the two MAML variants (std = 0.0431 and 0.0450 for 5 and 10 gradient steps respectively).

The experiments revealed clear difference in performance between the Fine-tuned Frequent Hitters and the Baseline Frequent Hitters. The addition of a simple fine-tuning step resulted in a considerable increase in the mean DAUPRC

score, from 0.0013 to 0.0747. This suggests that while pre-training on different tasks does provide a good starting point, it is the task-specific tuning which gives the model its practical prediction.

Compared to the more complex MAML algorithm, the proposed model was not only competitive, but even performed better on the ROC AUC metric. Although MAML with 10 inner-loop gradient steps performed better on the DAUPRC metric, Fine-tuned Frequent Hitters is still a strong competitor considering that it is much simpler and more straightforward. Furthermore, it is more stable, as evidenced by its smaller standard deviation across the 10 random seeds (see *Table 2*). The implication of these results is clear: the transfer learning strategy embodied by the Fine-tune Frequent Hitters model, although simpler, can compete with or even outperform a more complex method like MAML, making it a more suitable alternative in this few-shot learning setting.

5 Conclusion

This thesis is conducted to answer an important research question: Can a simple fine-tuned Frequent Hitters model compete with, or even outperform, the performance of a state-of-the-art meta-learning algorithm like MAML for few-shot learning in drug discovery? The extensive experiments and analysis presented in the above sections provide a clear affirmative answer. The findings of this research indicate that an effective transfer learning approach, which involves pre-training a Frequent Hitters model followed by fine-tuning it on specific tasks, is not only competitive but can even outperform more complex approaches like MAML. The improved performance of the Fine-tuned Frequent Hitters model on the ROC AUC metric indicates the practical robustness of adapting generalized knowledge to new tasks with only a few examples. Although MAML was able to perform better on the DAUPRC metric when performing more inner gradient steps, its computational complexity may not always be justified, since a more direct and simple approach can yield similar or even better results. The practical impact on drug discovery can be significant as it validates a strong yet relatively simple and generalizable method that can be applied to real-world projects that contribute to the identification of promising candidate molecules.

Although this work provides valuable information, it is important to acknowledge its limitations, which can illuminate paths for future study. The findings are drawn from a single benchmark dataset; future research should seek to replicate these findings across a broader range of datasets. The molecular representations were limited to standard fingerprints and descriptors, and the use of more sophisticated features, such as those derived from graph neural networks, might improve the performance of all models [23, 24].

In conclusion, this thesis successfully demonstrates that a relatively simple fine-tuned frequent hitters model can be more effective compared to a state-of-the-art

meta-learning algorithm like MAML in the field of few-shot learning in drug discovery. The output of the Fine-tuned Frequent Hitters model offers a practical, robust, and highly effective approach to address the problem of data scarcity in the long and costly process of finding new drugs.

6 Summary Q&A

1. **What is the central research question of this thesis?**

   The central research question is whether a straightforward transfer learning approach, the Fine-tuned Frequent Hitters (FTFH) model, can compete with the state-of-the-art meta-learning algorithm MAML for few-shot learning in drug discovery. This thesis addresses the challenge of data scarcity in drug discovery by evaluating a simpler and less computationally complex alternative to advanced meta-learning. The thesis provides a clear affirmative answer that validates the effectiveness of the simple fine-tuning strategy.

2. **What are the main findings of this thesis?**

   The Fine-tuned Freuent Hitters model achieved the highest ROC AUC score of 0.7136 ± 0.0290 among all tested models, outperforming both MAML variants and the Random Forest baseline, indicating a better discriminative ability. Although MAML with 10 inner steps had a slightly higher mean DAUPRC score (0.0798 vs. 0.0747), the Fine-tuned Frequent Hitters model was more stable, with a lower standard deviation. These results suggest that a well-implemented, simpler fine-tuning strategy can be a competitive performer and a robust alternative to complex meta-learning.

3. **Why is the Fine-tuned Frequent Hitters approach significant for drug discovery?**

   Drug discovery is costly and limited in data, which hinders conventional machine learning methods. The Fine-tuned Frequent Hitters model provides a practical, robust, and simpler method that successfully addresses the few-shot learning problem by effectively adapting generalized knowledge to new drug tasks with minimal data. This provides a strong alternative to the complex MAML algorithm, offering a simpler method to help identify promising drug candidate molecules in real-world projects.

7 References

[1] Arrowsmith, J. (2011). Phase ii failures: 2008-2010. Nature reviews drug discovery, 10(5).

[2] Schimunek, J., Friedrich, L., Kuhn, D., Rippmann, F., Hochreiter, S., and Klambauer, G. A generalized framework for embedding-based few-shot learning methods in drug discovery.

[3] Bendre, N., Marín, H. T., and Najafirad, P. (2020). Learning from few samples: A survey.

[4] Wang, Y., Yao, Q., Kwok, J. T., and Ni, L. M. (2020). Generalizing from a few examples: A survey on few-shot learning.

[5] Cherkasov, A., Muratov, E. N., Fourches, D., Varnek, A.; Baskin, I. I., Cronin, M., Dearden, J., Gramatica, P., Olah, M., Solovetskaya, O., Tropsha, A., Zakharov, A. V., Liu, Y. (2014). QSAR Modeling: Where Have You Been? Where Are You Going? J. Med. Chem., 57, 4977–5004.

[6] Wigh DS, Goodman JM, Lapkin AA. (2022). A review of molecular representation in the age of machine learning. WIREs Comput Mol Sci.; 12:e1603.

[7] Rogers, D. and Hahn, M. (2010). Extended-connectivity fingerprints. Journal of chemical information and modeling.

[8] Moriwaki, H., Tian, YS., Kawashita, N. et al. (2018). Mordred: a molecular descriptor calculator. J Cheminform 10, 4.

[9] Singh, A., Thakur, N., and Sharma, A. (2016). A review of supervised machine learning algorithms.

[10] Altae-Tran, H., Ramsundar, B., Pappu, A. S., Pande, V. (2017). Low Data Drug Discovery with One-Shot Learning.

[11] Chen, T., Kornblith, S., Norouzi, M., and Hinton, G. (2020). A simple framework for contrastive learning of visual representations. In International conference on machine learning, pages 1597–1607. PMLR.

[12] Antoniou, A. and Storkey, A. (2019). Assume, augment and learn: Unsupervised few-shot meta-learning via random labels and data augmentation.

[13] Vinyals, O., Blundell, C., Lillicrap, T., Wierstra, D., et al. (2016). Matching networks for one shot learning.

[14] Snell, J., Swersky, K., and Zemel, R. S. (2017). Prototypical networks for few-shot learning.

[15] Schimunek, J., Seidl, P., Friedrich, L., Kuhn, D., Rippmann, F., Hochreiter, S., and Klambauer, G. (2023). Context-enriched molecule representations improve few-shot drug discovery.

[16] Finn, C., Abbeel, P., and Levine, S. (2017). Model-agnostic meta-learning for fast adaptation of deep networks.

[17] Sebastian G. Rohrer and Knut Baumann. (2009). Maximum Unbiased Validation (MUV) data sets for virtual screening based on PubChem bioactivity data.

[18] Wu, Z., Ramsundar, B., Feinberg, E. N., Gomes, J., Geniesse, C., Pappu, A. S., Leskovec, J., Pande, V. (2018). MoleculeNet: a benchmark for molecular machine learning.

[19] Andrew P. Bradley. (1997). The use of the area under the ROC curve in the evaluation of machine learning algorithms.

[20] Qi, Q., Youzhi, L., Zhao, X., Shuiwang, J., Tianbao, Y. (2021). Stochastic Optimization of Areas Under Precision-Recall Curves with Provable Convergence.

[21] Saito, T., Rehmsmeier, M. (2015). The Precision–Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets.

[22] Breiman, L. (2001). Random Forests. *Machine Learning* **45**, 5-32.

[23] Hu, W., Liu, B., Gomes, J., Zitnik, M., Liang, P., Pande, V., Leskovec, J. (2020). Strategies for Pre-training Graph Neural Networks.

[24] Rong, Y., Bian, Y., Xu, T., Xie, W., Wei, Y., Huang, W., Huang, J. (2020). Self-Supervised Graph Transformer on Large-Scale Molecular Data.

A Appendix

A.1 Baseline FH: Details and hyperparameters

To implement the baseline frequent hitters method, the dataset is prepared by splitting it into training, validation, and test sets. 10 tasks are used for training, 3 tasks for validation, and 3 tasks for testing. All features are standardized using *StandardScaler* to ensure consistent scaling.

Data Loader: For the dataset handling, I define a $MoleculeDataset$ class that extends PyTorch's *Dataset* class. Each dataset instance is represented as $\mathcal{D} = \{(x_k, y_k)\}_{k=1}^{N}$ where $x_k \in \mathbb{R}^{2248}$ is the molecular feature vector and $y_k \in \{0, 1\}$ is the binary activity label. After defining the $MoleculeDataset$ class, I create a single Data Loader for the training dataset, treating each molecule-task pair as an independent training sample. This means that the same molecule can appear multiple times in the dataset, associated with different tasks, and each instance is treated independently. Using this approach, we do not differentiate between which task a molecule is associated with, but

instead, the model learns patterns from the entire dataset.

For validation and testing, task-specific datasets and loaders are created, allowing the model's performance to be evaluated on different tasks separately while keeping the training phase generalized across all molecule-task pairs. This separation ensures that validation and testing are done in a targeted manner, helping identify how well the model generalizes across different tasks.

Neural Network's Architecture: I define a simple feed-forward neural network architecture with ReLU activation function and dropout for regularization. The input layer size is determined by the number of features (2,248), and the output layer size is set to 1 for binary classification. The sizes of hidden layers are set to 64, 32, 16, respectively.

```
class BaselineFHNN(nn.Module):
def __init__(self, input_size, hs1, hs2, hs3, output_size):
    super(BaselineFHNN, self).__init__()
    self.fc1 = nn.Linear(input_size, hs1)
    self.fc2 = nn.Linear(hs1, hs2)
    self.fc3 = nn.Linear(hs2, hs3)
    self.fc4 = nn.Linear(hs3, output_size)
    self.relu = nn.ReLU()
    self.sigmoid = nn.Sigmoid()
    self.input_dropout = nn.Dropout(0.25)
    self.dropout = nn.Dropout(0.5)

def forward(self, x):
    x = self.input_dropout(x)
    x = self.dropout(self.relu(self.fc1(x)))
    x = self.dropout(self.relu(self.fc2(x)))
    x = self.dropout(self.relu(self.fc3(x)))
    x = self.sigmoid(self.fc4(x))
    return x
```

Code Block 1: Feed-forward Neural Network Architecture for the Baseline Frequent Hitters Model.

The model is then created, optimizing the parameters using *Adam* optimizer and using Binary Cross-Entropy Loss as the minimization criterion. For training, ten random seeds are set to ensure reproducibility and robustness, and the training process involves iterating over five epochs, adjusting the model's weights based on the calculated gradients. Different combinations of hyperparameters and model architectures (learning rate, batch size, hidden layers, etc.) were tried to achieve the best possible results (see *Table 3*).

Training and Validation: During the training phase, the model processes batches of features and labels from the training Data Loader. For each batch,

Hyperparameter	Explored Values
Number of hidden layers	1, 2, **3**, 4
Number of units per hidden layer	128-64-32, **64-32-16**, 32-16-8
Learning rate	0.1, **0.01**, 0.001, 0.0001, 0.00001
Optimizer	**Adam**, SGD
Batch size	512, 1024, 2048, 4096, **8192**
Activation function	**ReLU**, SELU
Input dropout	0.1, **0.25**
Dropout	0.3, **0.5**, 0.7

Table 3: Explored hyperparameters for the Baseline Frequent Hitters model. Bold values represent the best configuration

the model makes predictions, calculates the loss, and updates the model's parameters through back-propagation and gradient descent. After each training epoch, the model is switched to evaluation mode, where its performance is tested on validation datasets corresponding to different validation tasks. For each validation task, predictions and true labels are collected, and the performance is then evaluated using the ROC AUC score and the Mean DAUPRC score. These validation tasks are used to optimize the hyperparameters to achieve the best possible results.

Testing: During the testing phase, the trained model is evaluated on multiple separate test tasks once, which gives an unbiased estimate of the model's ability to generalize to new unseen tasks. For each test task, predictions and true labels are collected, and the performance is then evaluated using the ROC AUC score and the mean DAUPRC score. Finally, the average ROC AUC and DAUPRC scores are calculated in all test tasks, and the mean and standard deviation are calculated in all seeds.

A.2 Fine-tuned FH: Details and hyperparameters

Pre-training Phase: This phase is similar to the baseline frequent hitters method, where a feed-forward neural network is trained on the aggregated dataset of 10 training tasks. This initial stage creates a pre-trained model with a generalized knowledge of structure-activity relationships, which serves as the foundation for the second phase. The network's architecture used for per-training consists of one hidden layer of size 128.

```
class FineTunedFHNN(nn.Module):
def __init__(self, input_size, hs1, output_size):
    super(FineTunedFHNN, self).__init__()
    self.fc1 = nn.Linear(input_size, hs1)
    self.fc2 = nn.Linear(hs1, output_size)
    self.relu = nn.ReLU()
    self.sigmoid = nn.Sigmoid()
```

```
        self.input_dropout = nn.Dropout(0.25)
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        x = self.input_dropout(x)
        x = self.dropout(self.relu(self.fc1(x)))
        x = self.sigmoid(self.fc2(x))
        return x
```

Code Block 2: Feed-forward Neural Network Architecture for Fine-tuned Frequent Hitters Model.

Fine-tuning Phase: This phase adapts the general knowledge acquired by the pre-trained model to the specific characteristics of a new, unseen task. For each of the three test tasks, the fine-tuning process begins by creating a 5-shot support set, which is formed by randomly selecting 5 positive (active) and five negative (inactive) molecules from the task's dataset, and the remaining molecules forms the query set which is used for final evaluation. Then, the fine-tuning process begins by training the model exclusively on the 5-shot support set for 300 episodes using *Adam* optimizer and a fine-tuning learning rate of 0.0001. This intensive training on a very small dataset allows the model to adjust its parameters to the specific characteristics of the new task without ignoring the features learned during pre-training. After the fine-tuning process is complete for one task, the resulting fine-tuned model is set to evaluation mode and its prediction performance is then measured on the task's query set using the ROC AUC and Mean DAUPRC metrics. The scores of both metrics are then averaged across all test tasks and all random seeds, to provide a robust measure of the model's generalization capability in a few-shot setting and the final results are reported.

Hyperparameter	**Explored Values**
Number of hidden layers	**1**, 2, 3
Number of units per hidden layer	16, 32, 64, **128**, 256
Pre-training Learning rate	0.01, 0.001, 0.0001, **0.00001**
Fine-tuning learning rate	0.0, 0.001, **0.0001**, 0.00001
Number of pre-training epochs	**3**, 5, 10, 20, 50
Number of fine-tuning episodes	10, 50, 100, **300**, 500, 1000
Optimizer	**Adam**, SGD
Batch size	512, 1024, **2048**, 4096, 8192
Activation function	**ReLU**, SeLU
Input dropout	0.1, **0.25**
Dropout	0.3, **0.5**, 0.7

Table 4: Explored hyperparameters for the Fine-tuned Frequent Hitters model. Bold values represent the best configuration

A.3 MAML: Details and hyperparameters

Data Handling for Meta-Learning: Unlike the single Data Loader that is used for baseline training, the MAML meta-training process requires a more dynamic data handling strategy. The training data from the 10 training tasks is structured to be sampled as a sequence of different few-shot learning tasks. In each meta-training episode, a batch of 4 tasks is randomly sampled. For each sampled task, a 5-shot support set is created for the inner-loop adaptation by selecting five positive and five negative examples randomly, and the remaining examples from that task forms the query set for the outer-loop meta update.

Neural Network's Architecture: I define a similar feed-forward neural network to the Frequent Hitters model. However, a key modification for MAML is the inclusion of a *functional_forward* method, which allows the network to perform a forward pass using explicit weights passed as an argument, rather than using the model's internal state. This is important for the meta-update step, where the loss must be calculated based on the temporary, adapted weights from the inner loop. The input layer size is 2,248, the hidden layer is of size 128, and the size of the output layer is 1 for binary classification.

```
class MAMLNN(nn.Module):
def __init__(self, input_size, hs1, output_size):
    super(MAMLNN, self).__init__()
    self.fc1 = nn.Linear(input_size, hs1)
    self.fc2 = nn.Linear(hs1, output_size)
    self.relu = nn.ReLU()
    self.sigmoid = nn.Sigmoid()
    self.input_dropout = nn.Dropout(0.25)
    self.dropout = nn.Dropout(0.5)

def forward(self, x):
    x = self.input_dropout(x)
    x = self.dropout(self.relu(self.fc1(x)))
    x = self.sigmoid(self.fc2(x))
    return x

def functional_forward(self, x, weights):
    x = nn.functional.linear(x,
    weights["fc1.weight"], weights["fc1.bias"])
    x = self.relu(x)
    x = nn.functional.linear(x,
    weights["fc2.weight"], weights["fc2.bias"])
    x = self.sigmoid(x)
    return x
```

Code Block 3: Feed-forward Neural Network Architecture for MAML.

Meta-Training: The model's parameters are optimized using a nested loop structure over 10 meta-training episodes.

- **Inner Loop:** For each sampled task, the loss is calculated on the 5-shot support set, and the gradients are computed with *create_graph=True* to maintain the computational graph. These gradients are used to calculate a temporary set of weights by performing five gradient descent steps with an inner learning rate of 0.1.
- **Outer Loop:** The temporary weights calculated in the inner loop are then used to evaluate the model's performance on the task's query set via the *functional_forward* method. The loss from this evaluation is aggregated across all tasks in the meta-batch to form a single meta-loss. Finally, a meta-update is performed by calling *backward()* on the meta-loss and taking a step with the *Adam* meta-optimizer with a meta learning rate of 0.0001, which updates the original model parameters θ.

Testing: During the testing phase, the final meta-trained model is used as a starting point for each of the three test tasks. For each test task, a copy of the model is adapted by performing 5 gradient descent steps (or 10 steps) on the task's support set. After the adaptation is complete, the final specialized model is evaluated on the task's query set using the ROC AUC and Mean DUAPRC scores, and the final results are reported.

Hyperparameter	**Explored Values**
Number of hidden layers	**1**, 2, 3
Number of units per hidden layer	16, 32, 64, **128**, 256
Meta learning rate	0.1, 0.01, 0.001, **0.0001**, 0.00001
Inner learning rate	**0.1**, 0.01, 0.001, 0.0001, 0.00001
Number of episodes	5, **10**, 20, 50, 100
Number of gradient steps	1, **5**, **10**
Tasks per meta-batch	2, **4**, 10
Optimizer	**Adam**, SGD
Activation function	**ReLU**, SeLU
Input dropout	0.1, **0.25**
Dropout	0.3, **0.5**, 0.7

Table 5: Explored hyperparameters for the MAML model. Bold values represent the best configuration